

AI Assisted Coding

Assignment – 18.3

Name: K. Sai Krishna Reddy

Rollno: 2303A52305

Batch:45

Task 1 – Public Transport Route Fare API

Task:

Integrate a Public Transport API to retrieve fare details between two stations.

Instructions:

- Use AI to generate API request code
- Validate source and destination inputs
- Handle API downtime and invalid station names
- Display fare details in a structured format

Expected Output:

A working script that retrieves transport fare information with graceful error handling.

The screenshot shows a Visual Studio Code editor with a Python file named `Task-2.py`. The code is a currency converter that takes an input amount in INR, converts it to USD, EUR, and GBP, and displays the results. It includes error handling for invalid inputs and connection errors. The terminal output shows the program running and displaying the conversion results for 1,000 INR.

```
138 def main():
139
140     while True:
141         try:
142             amount_input = input("\nEnter amount in INR (or 'quit' to exit): ").strip()
143
144             if amount_input.lower() == 'quit':
145                 print("Thank you for using the Currency Converter!")
146                 break
147
148             try:
149                 amount = float(amount_input)
150             except ValueError:
151                 print("Please enter a valid number.")
152                 continue
153
154             conversions = api.convert_currency(amount)
155             display_conversion_results(amount, conversions, api.last_update)
156
157         except (ValueError, ConnectionError) as e:
158             print(f"Error: {e}")
159             print("Please try again.")
160         except KeyboardInterrupt:
161             print("\nExiting Currency Converter...")
162             break
163
164     if __name__ == "__main__":
165         main()
```

Terminal Output:

```
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-18.3> cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-18.3'; & 'c:\Users\K*
atku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy
\launcher' '56457' '-' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-18.3\Task-2.py'
Converting ₹1,000 INR...

=====
CURRENCY EXCHANGE RESULTS
=====
Amount in INR: ₹1,000.00
=====
USD: $10.70
EUR: €9.08
GBP: £7.90
=====
```

Task 3 – GitHub Repository Info Fetcher

Task:

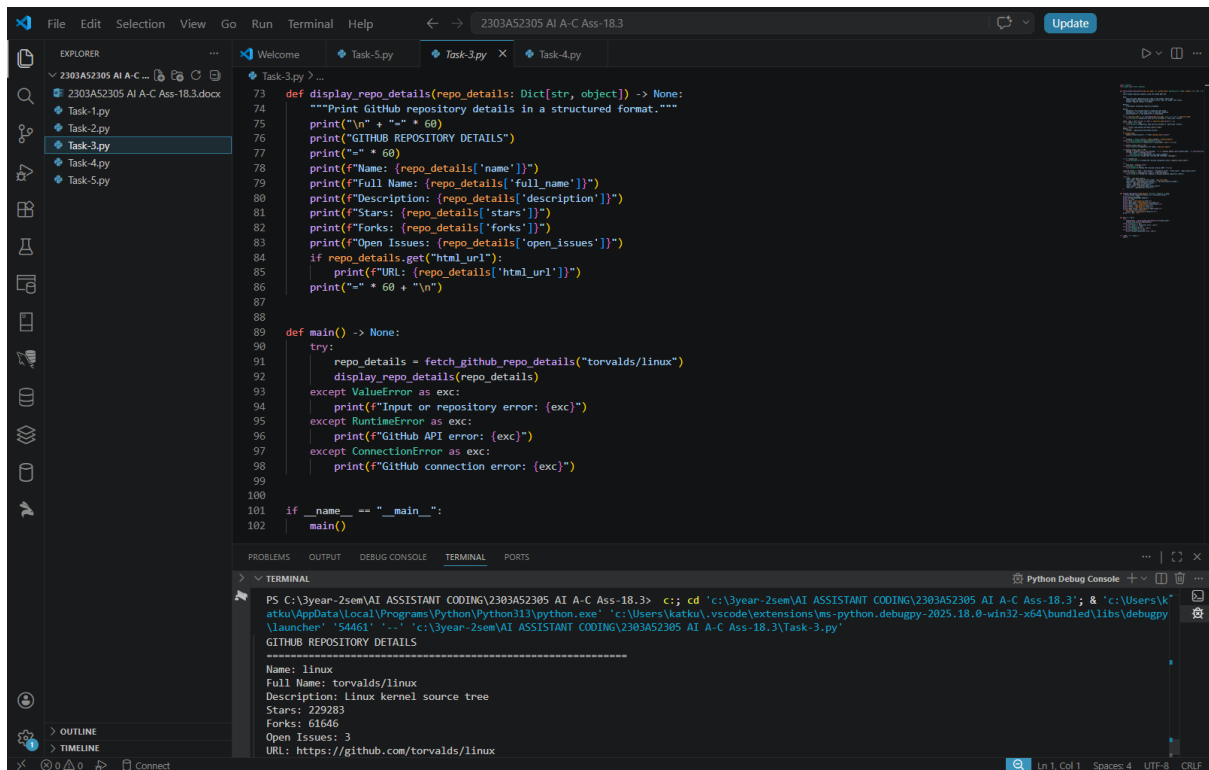
Connect to the GitHub API to fetch details of a repository (e.g., stars, forks, issues).

Instructions:

- Prompt AI to write a Python function to send GET requests to GitHub API.
- Handle rate limit errors, 404 (repo not found), and invalid input.
- Display repository name, description, stars, forks, and open issues.

Expected Output:

- A Python program that retrieves and displays GitHub repository information with robust error handling.



The screenshot shows a Visual Studio Code editor with a Python file named `Task-3.py`. The script defines a function `display_repo_details` that takes a dictionary of repository details and prints them in a structured format. It also includes a `main` function that fetches details for the 'torvalds/linux' repository and handles various exceptions. The terminal window at the bottom shows the command to run the script and the resulting output, which displays the repository details for 'torvalds/linux'.

```
73 def display_repo_details(repo_details: Dict[str, object]) -> None:
74     """Print GitHub repository details in a structured format."""
75     print("\n" + "-" * 60)
76     print("GITHUB REPOSITORY DETAILS")
77     print("-" * 60)
78     print(f"Name: {repo_details['name']}")
79     print(f"Full Name: {repo_details['full_name']}")
80     print(f"Description: {repo_details['description']}")
81     print(f"Stars: {repo_details['stars']}")
82     print(f"Forks: {repo_details['forks']}")
83     print(f"Open Issues: {repo_details['open_issues']}")
84     if repo_details.get("html_url"):
85         print(f"URL: {repo_details['html_url']}")
86     print("\n" + "-" * 60 + "\n")
87
88
89 def main() -> None:
90     try:
91         repo_details = fetch_github_repo_details("torvalds/linux")
92         display_repo_details(repo_details)
93     except ValueError as exc:
94         print(f"Input or repository error: {exc}")
95     except RuntimeError as exc:
96         print(f"GitHub API error: {exc}")
97     except ConnectionError as exc:
98         print(f"GitHub connection error: {exc}")
99
100
101 if __name__ == "__main__":
102     main()
```

```
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-18.3> cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-18.3'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '54461' '-.' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-18.3\Task-3.py'
GITHUB REPOSITORY DETAILS
-----
Name: linux
Full Name: torvalds/linux
Description: Linux kernel source tree
Stars: 229283
Forks: 61646
Open Issues: 3
URL: https://github.com/torvalds/linux
```

Task 4 – Real-Time Application: News Headlines Aggregator

Scenario:

Build a News Aggregator using a free News API.

Requirements:

- Fetch top 5 headlines for a given category (sports, technology, health).
- Handle errors like invalid category, missing API key, and request failures.
- Display headlines in a user-friendly numbered list format.
- Include a retry mechanism if the first request fails.

Expected Output:

- A working Python script that retrieves and displays news headlines with proper error handling

The screenshot shows a VS Code editor with a Python file named `Task-4.py`. The code is a news aggregator script that takes user input for a news category and fetches headlines from a simulated API. The terminal output shows the script running successfully, displaying the top 3 headlines for the 'SPORTS' category.

```
def main() -> None:
    # Interactive mode
    print("\n" + "-" * 80)
    print("INTERACTIVE NEWS AGGREGATOR")
    print("-" * 80)

    while True:
        try:
            category = input(
                "\nEnter news category (sports/technology/health) or 'quit' to exit: "
            ).strip()

            if category.lower() == "quit":
                print("Thank you for using the News Aggregator!")
                break

            headlines = aggregator.fetch_headlines(category, max_articles=5)
            aggregator.display_headlines(headlines, category)

        except ValueError as exc:
            print(f"Input Error: {exc}")
        except RuntimeError as exc:
            print(f"API Error: {exc}")
        except KeyboardInterrupt:
            print("\nExiting...")
            break

if __name__ == "__main__":
    main()
```

Terminal Output:

```
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-18.3> cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-18.3' & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '63381' '-.' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-18.3\Task-4.py'
TOP 3 HEADLINES - SPORTS
1. Football Championship Live Updates
   Sources: Sports Hub
   Published: 2026-04-14T10:00:00Z
   Description: Latest updates from the international football championship
   URL: https://example.com/sports1
```

Task 5 -COVID-19 Statistics API Integration

Task:

Connect to a COVID-19 Statistics API to fetch country-wise COVID data.

Instructions:

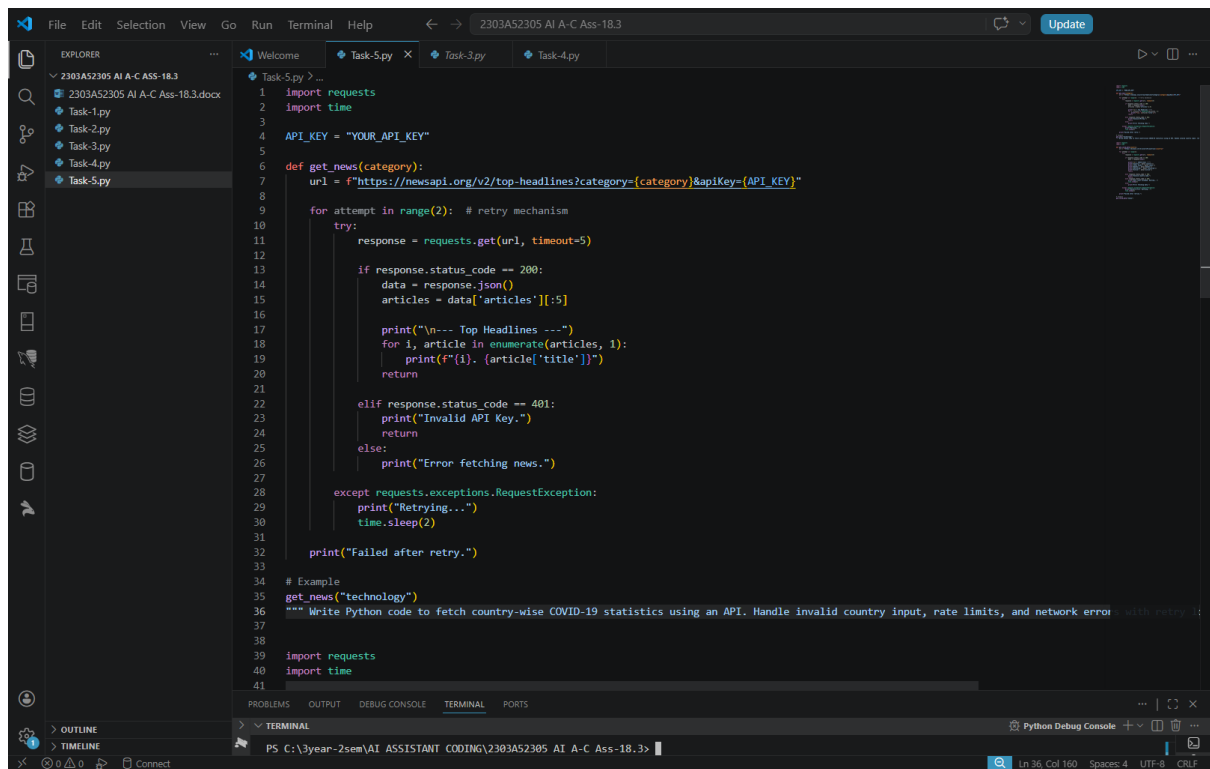
- Use AI to generate Python code using the requests library
- Accept country name as user input
- Fetch key statistics such as:
 - o Total confirmed cases
 - o Total deaths
 - o Total recovered cases
 - o Active cases
- Handle API errors including:
 - o Invalid country name
 - o API rate-limit exceeded

o Network or server failures

- Implement retry or wait logic when rate limits are hit

Expected Output:

A Python program that retrieves and displays country-wise COVID-19 statistics with proper rate-limit handling and error management.



The screenshot shows a Visual Studio Code editor window with a Python file named 'Task-5.py'. The code implements a function to fetch news from a specific category using the NewsAPI. It includes a retry mechanism for handling network or server failures. The code is as follows:

```
1 import requests
2 import time
3
4 API_KEY = "YOUR_API_KEY"
5
6 def get_news(category):
7     url = f"https://newsapi.org/v2/top-headlines?category={category}&apiKey={API_KEY}"
8
9     for attempt in range(2): # retry mechanism
10        try:
11            response = requests.get(url, timeout=5)
12
13            if response.status_code == 200:
14                data = response.json()
15                articles = data['articles'][:5]
16
17                print("\n--- Top Headlines ---")
18                for i, article in enumerate(articles, 1):
19                    print(f"{i}. {article['title']}")
20                return
21
22            elif response.status_code == 401:
23                print("Invalid API Key.")
24                return
25            else:
26                print("Error fetching news.")
27
28        except requests.exceptions.RequestException:
29            print("Retrying...")
30            time.sleep(2)
31
32    print("Failed after retry.")
33
34 # Example
35 get_news("technology")
36 """ Write Python code to fetch country-wise COVID-19 statistics using an API. Handle invalid country input, rate limits, and network errors with retry logic. """
37
38 import requests
39 import time
40
41
```

The terminal at the bottom shows the command prompt with the file path: `PS C:\3year-2sem\AI ASSISTANT CODING\2303AS2305 AI A-C Ass-18.3>`. The status bar at the bottom indicates the file is at line 36, column 160, using UTF-8 encoding.